

# EE 513 — Setup and First Captures

Crow Crossman

October 6, 2026

**Repository:** <https://github.com/acrossman/ee513>

## Environment

### 1. Course environment smoke test

*Build the course environment from the two files on the setup page and run the smoke test notebook. Deliverable: the version block the smoke test prints, showing Python, NumPy, and OpenCV versions.*

```
[PASS] Python 3.10 or newer found 3.12
[PASS] numpy 2.5.3
[PASS] OpenCV major version is 5 found 5.0.0
```

### 2. Project 1 repository

*Create a public GitHub repository for Project 1, with a README naming the camera you intend to characterize and the app you will use to capture raw.*

*Deliverable: the repository URL.*

<https://github.com/acrossman/ee513>

## Confirm Your Camera Writes Real Raw

### 3. rawpy metadata

*Capture one raw photograph of anything, load it with `rawpy`, and print `raw_type`, the shape and `dtype` of `raw_image_visible`, `raw_pattern`, `color_desc`, `black_level_per_channel`, and `white_level`.*

*Deliverable: the printed output.*

```
raw_type: RawType.Flat
-----
raw_image_visible: [[ 67 67 66 ..., 100 93 102]
 [ 70 67 69 ..., 75 97 73]
 [ 67 70 67 ..., 104 92 100]
 ...,
 [ 92 72 91 ..., 69 83 70]
 [ 82 93 80 ..., 84 78 83]
```

```
[ 88 73 90 ..., 69 82 68]]
shape: (3472, 4624)
dtype: uint16
-----
raw_pattern: [[0 1]
[3 2]]
-----
color_desc: b'RGBG'
-----
black_level_per_channel: [64, 64, 64, 64]
-----
white_level: 1023
```

#### 4. Bayer mosaic verdict

*State in one sentence whether you have a Bayer mosaic. You want `RawType.Flat`, a two-dimensional `raw_image`, and a `2x2 raw_pattern`. A three-dimensional array or a `raw_pattern` of `None` means the camera has already demosaiced the file and it will not work for Project 2.*

*Deliverable: a one-sentence verdict, and the CFA layout as four letters, for example `RGGB` or `BGGR`.*

I have a Bayer Mosaic as confirmed by the `raw_type` output of `RawType.Flat`, confirmation of two-dimensions from raw image shape outputting (3472, 4624), a `2x2 raw_pattern`, and a CFA layout of 'RGBG' based on the `color_desc`, `raw_pattern`, and documentation from <https://letmaik.github.io/rawpy/api/rawpy.RawPy.html>.

#### 5. If not a Bayer mosaic

*If you do not have a Bayer mosaic, say which app and settings you tried and what you will do next. Apple ProRAW is the usual culprit on iPhone; Open Camera with the Camera2 API and "JPEG and DNG (RAW)" is the usual fix on Android. Email me this week if you are stuck rather than after the deadline.*

*Deliverable: a written statement, or "not applicable" if item 4 succeeded.*

Not applicable.

### What the Ordinary Photo Path Threw Away

#### 6. Raw vs. JPEG/HEIC comparison

*Capture the same scene twice without moving the camera, once as raw and once as an ordinary JPEG or HEIC. Report the file size, bit depth, and number of channels of each.*

*Deliverable: a small table with both files.*

| File        | Size (bytes) | Bit depth | Channels | Shape           |
|-------------|--------------|-----------|----------|-----------------|
| Raw (.dng)  | 32,151,936   | uint16    | 1        | (3472, 4624)    |
| JPEG (.jpg) | 1,209,805    | uint8     | 3        | (4624, 3472, 3) |

## 7. Zoomed raw crop

*Display a region of about 30 by 30 pixels of the raw data, zoomed so that individual pixels are visible as blocks.*

*Deliverable: the figure, and a sentence identifying what the checkerboard texture is. It is not noise.*

Below is a 30 by 30 pixel section of the raw data from my raw image capture. I focused on a corner of the image that was fundamentally a white section of the photo. The pixels show the Bayer mosaic pattern in the 'Greens' color map for matplotlib.

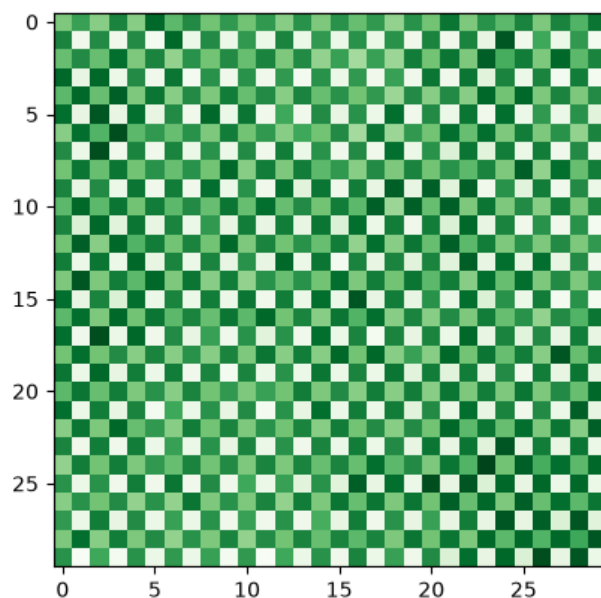


Figure 1: 30x30 pixel region of the raw data.

## 8. Irreversible JPEG pipeline steps

*Name three things the camera's own pipeline did to the JPEG that cannot be undone afterwards, and say for each one why it is irreversible.*

*Deliverable: a short written answer, three or four sentences.*

The camera pipeline performed a number of irreversible changes. Based on the outputs from task 6 we can see that the size of the file has been reduced, the bit depth has been reduced, and color channels have been created. Based on readings from Bhandari, Kadambi, and Raskar (section 2.2.4) this reflects the process of taking a raw image file and performing a handful of corrections/alterations:

1. **Demosaicing.** First the image is demosaiced, using the monochromatic pixel values from the mosaic to interpolate each color channel. Since these values are based on interpolated estimates we cannot rebuild the original mosaic with perfect accuracy from the post-processed jpeg alone.
2. **Bit depth reduction.** The bit depth has changed with the actual sensor values of the colors started as potentially 16 bits (it was unclear to me how to confirm the number of bits used but `dtype` returned `uint16`), but have been quantized to 8 bits (values between 0–255). Since the

values have been quantized to 8 bits we have no way of knowing whether a min or max value is more or less in additional bit depth.

3. **Compression.** The file size has been compressed by 96%, but I could not confirm the criteria as to why this level of compression is deemed appropriate. Even if this reduction is a nearly 100% approximation there is still uncertainty in trying to reverse any discarded information.
4. **White balance and gamma correction.** I did not confirm this through programming but the reading suggests that other standard corrections include denoising, white balance, as well as gamma balance. Once these corrections are performed there is no stored data to allow for reversing as far as I know.

All of these result in a loss of information that cannot be reconstructed into full fidelity of the original raw value.

## First Calibration Captures

### 9. Measured square size

*Print the calibration target at 100% scale and tape it flat to card or board. Measure one square with a ruler and report the measured size in millimetres.*

*Deliverable: the measured square size, and one sentence on why you will use your measured value rather than the nominal 20.0 mm.*

I thought I had printed the pdf at 100% but when I measured the reference bar it came out to 96 mm and the sides of the squares came out to 19 mm each. I will use the measured 19 mm because the square size is what gives the calibration its physical scale.

### 10. Calibration captures

*Capture at least 15 photographs of the target at a variety of angles, distances, and positions in the frame, including several where the board is tilted well away from parallel and several where it sits near a corner of the frame.*

*Deliverable: a contact sheet of your captures.*

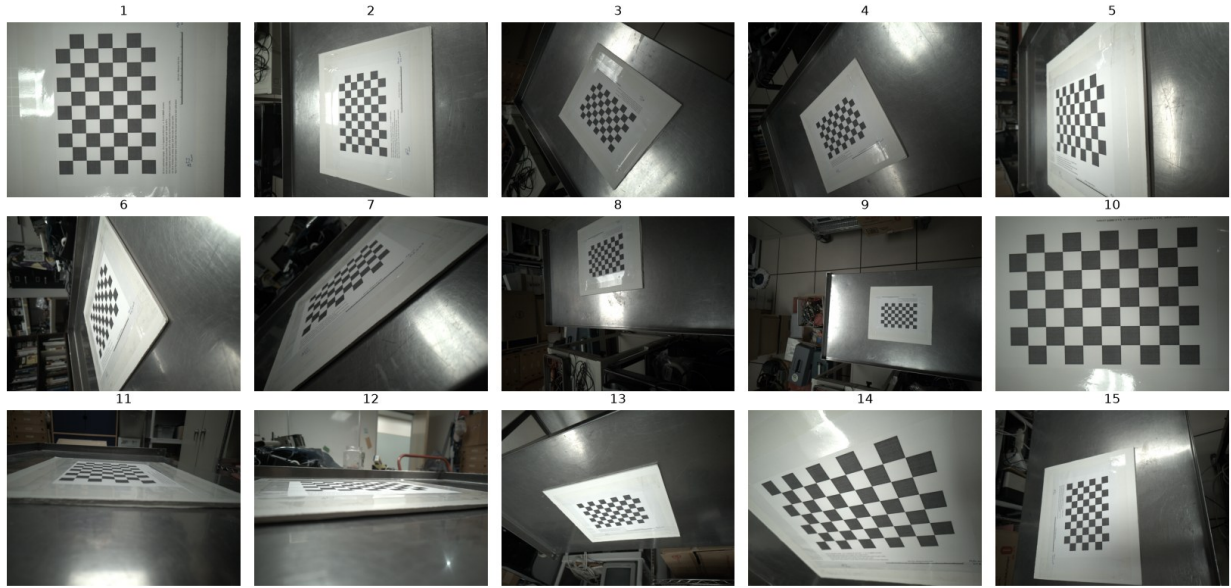


Figure 2: Contact sheet of the 15 calibration captures, numbered in capture order and shown in the sensor's native orientation.

## 11. Chessboard detection

*Run `cv2.findChessboardCorners` with the pattern size  $(9, 6)$  on every capture and report how many succeeded.*

*Deliverable: the count, and one image with the detected corners drawn on it.*

The function detected 11 out of 15 images failing on severely tilted frames.

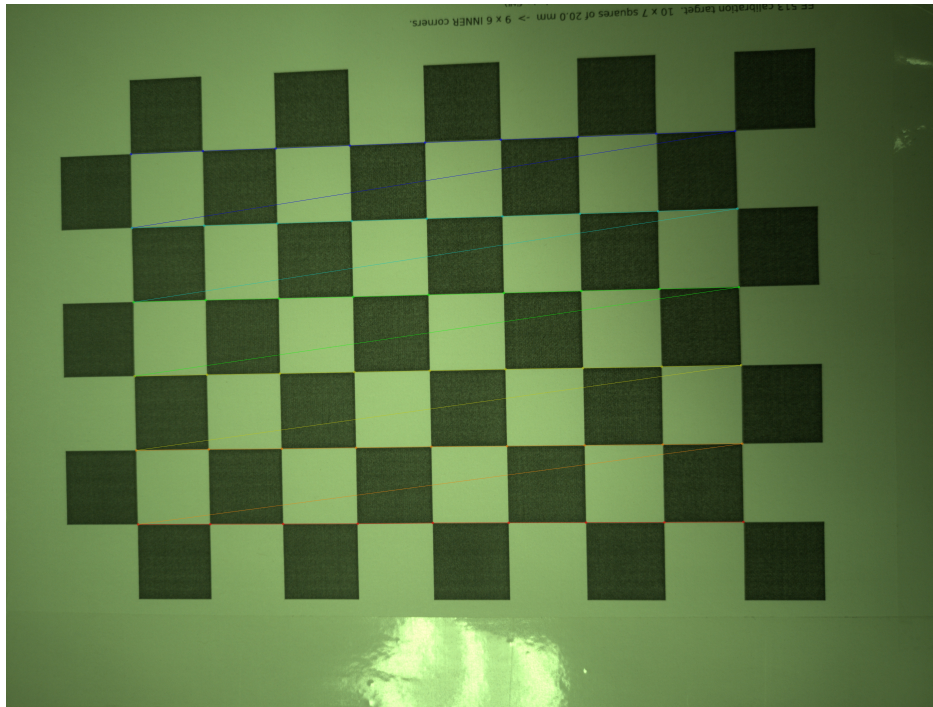


Figure 3: Detected  $9 \times 6$  inner corners drawn by `cv2.drawChessboardCorners`.

## 12. Calibration RMS reprojection error

Pass the successful detections to `cv2.calibrateCamera` and report the RMS reprojection error it returns. A rough number is fine here; Project 1 is where you will interpret it.

*Deliverable: the RMS reprojection error, and one sentence on what it means and what units it is in.*

```
Not detected: IMG_20261002_155655.dng
Not detected: IMG_20261002_155647.dng
Not detected: IMG_20261002_155624.dng
Not detected: IMG_20261002_155711.dng
11 of 15
0.930455325433955
```

The reprojection error came to 0.93 pixels representing the distance, in pixels, between the corners detected in each photo and where the known board points land when projected through the camera model. The calibration chooses the camera matrix, distortion coefficients, and each photo's pose so that this distance is as small as possible, and the RMS error is what remains.

## Reflections

### 13. What did you learn?

*What did you learn from this assignment?*

*Deliverable: a written response.*

For this assignment I found the reading from Bhandari, Kadambi, and Raskar incredibly helpful. It clarified a lot of ideas including concepts like ISO settings, gamma balance, and the digital image processing pipeline in general. The documentation from OpenCV on their tools was also helpful in reinforcing the calibration theory.

#### 14. Questions for the instructor

*What questions, if any, do you have for me after completing this assignment?*

*Deliverable: a written response.*

I don't have any particular questions at present since it is early in the term. I think that some of my broader questions will be answered as we progress, but I will make sure to put them out there as they come up.

#### Use of AI Tools

*On AI tools. Use them freely on this and every exercise. If you do, add two sentences at the end saying what you used them for and how you checked that the output was right. That habit is the one I will ask about all term, and it is what the projects grade.*

AI tools (Claude by Anthropic) were used in the composition of this report. The tools were used for organizing LaTeX code in addition to simple scripts. The author of this report reviewed any code used, and often discarded it in favor of some good old fashioned home rolled. I cannot overstate how much simpler it often is to use code provided by the designers and their documentation. Sometimes the tools were used to act as a tutor on concepts or to test the knowledge of the author.